

P1208R1

Adopt `source_location` from Library Fundamentals V3 for C++20

Draft Proposal, 2018-09-21

This version:

https://rawgit.com/cor3ntin/CppProposals/master/merge_source_location/P1208.html

Authors:

[Corentin Jabot](#)

[Robert Douglas](#)

Audience:

LEWG, LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes that `source_location` (from Library Fundamentals V3) be adopted into the C++20 standard

Table of Contents

- 1 **Proposal**
- 2 **A few design changes**
 - 2.1 Enforcing a size for `source_location` objects ?
 - 2.2 Removing `current()` altogether ?
- 3 **User-constructed `source_location`**
- 4 **Proposed Wording**

5 Feature macro

6 Acknowledgments

References

Informative References

§ 1. Proposal

This paper proposes that `source_location` from Library Fundamentals V3 [\[N4758\]](#) be adopted into the C++20 standard.

`source_location` has been successfully implemented without difficulties in GCC, and Clang implementers almost completed their implementation of it. As described by the original paper, it's a necessary feature to implement logging without requiring macros, one that cannot be portably implemented by non-standard libraries because it requires compiler support.

`source_location` has been very favorably received by both EWG and LEWG [\[n4129\]](#), [\[n4417\]](#) and has been part of library fundamentals v2 [\[n4562\]](#) and v3 [\[N4758\]](#) since 2015 without changes to exposed interface.

Note: A proposal [\[P0555R0\]](#) in 2017 to make use of `string_view` in `source_location` has been withdrawn by their authors once `string_view` gained a constexpr constructor, allowing the use of `source_location` in constexpr contexts.

§ 2. A few design changes

During an early review of this papers, some interest in a few design changes arose:

§ 2.1. Enforcing a size for `source_location` objects ?

No consensus in San Diego

`source_location` as currently implemented by Clang and GCC is a structure encapsulating all its members (file, function, line, column), and so its size is roughly `3 * sizeof(void*)`.

A note states:

[Note: The intent of `source_location` is to have a small size and efficient copying. — end note]

However, there seems to be some interest for having `source_location` be guaranteed `sizeof(void*)`. Notably, reviewers wonder if `source_location` should be embedded in a yet to be proposed `std::error`. Herb Sutter and Niall Douglas pointed out that `std::error` is meant to handle recoverable errors, rather than to communicate an error to developers (see Herb Sutter’s talk on error handling CppCon 2018). And while we agree strongly with this sentiment, it’s worth noting that a subset of the C committee is interested in having source information in whatever deterministic exception handling they are working towards;

While, as pointed out by Niall, it’s unlikely the two languages will have directly interoperable mechanisms, having a guaranteed size for `source_location` might make such interoperability easier should a need for it arise.

Barring that, `source_location` is an error reporting tool targeted at human developers, and the cost of any such error reporting system is many orders of magnitude larger than copying 3 pointers around. We do not think there is a strong incentive to guarantee the size of that type or restricting the way it is implemented.

For completeness, such modification could be achieved by adding one level of indirection:

```
struct __source_location_data {  
    /* implementation defined */  
};  
struct source_location {  
private:  
    const __source_location_data* __data;  
};
```

Alternatively, `source_location` could return a const reference:

```
struct source_location {  
    constexpr const source_location & current() noexcept;  
};
```

The authors strongly prefer the first solution as we think retaining value semantics is important. It is also important to note that, while not implemented, the first solution has been considered and is deemed realistically implementable by both GCC and Clang developers (Jonathan Wakely, Ericnie Fiselier).

§ 2.2. Removing `current()` altogether ?

No consensus in San Diego

If LEWG elects to keep value semantics, the authors would like to propose that the `current` function is removed, since, in its current form, `source_location` has a default constructor that has no meaningful use. Besides being harder to misuse, using the constructor of the type to acquire its value is also significantly less verbose.

```
void log(auto && message, std::source_location sc = std::source_location::current());  
//or  
void log(auto && message, std::source_location sc = {});
```

§ 3. User-constructed `source_location`

In San Diego, it was suggested to add the following constructor

```
constexpr source_location(uint_least32_t line,  
                           uint_least32_t column,  
                           const char* filename,  
                           const char* function);
```

So that `source_location` can be constructed by users and other libraries (stacktrace ?) However, `constexpr`, to the best of our understanding doesn't not guarantee that `filename` and `function` will live past the call to that constructor.

`filename` and `function` would have to be copied (at compile time) and it isn't clear whether it would be possible to copy them in read only memory.

Therefore we suggested that this could be the subject of a separate paper to explore these lifetime issues

§ 4. Proposed Wording

The following wording correspond to the wording as in [\[N4758\]](#) (Library fundamental), with the exception of making `source_location::current()` an immediate function as decided by LEWG in San Diego.

Notes To The Editor:

The `<source_location>` header in Library fundamental is part of a "Reflection" library that does not yet exist and may need to be created or be placed elsewhere.

The `<source_location>` header should be added to the list of freestanding headers

11.1 Header `<source_location>` synopsis [reflection.src_loc.syn]

```
namespace std {
    struct source_location {

        static constexpr source_location current() noexcept;

        constexpr source_location() noexcept;

        constexpr uint_least32_t line() const noexcept;
        constexpr uint_least32_t column() const noexcept;
        constexpr const char* file_name() const noexcept;
        constexpr const char* function_name() const noexcept;
    };
} // namespace std
```

[*Note*: The intent of `source_location` is to have a small size and efficient copying. — *end note*]

11.2.1 `source_location` creation [reflection.src_loc.creation]

```
static constexpr source_location current() noexcept;
```

Returns:

When invoked by a function call (C++17 §8.2.2) whose *postfix-expression* is a (possibly parenthesized) *id-expression* naming `current`, returns a `source_location` with an implementation-defined value. The value should be affected by `#line` (C++17 §19.4) in the same manner as for `__LINE__` and `__FILE__`. If invoked in some other way, the value returned is unspecified.

Remarks:

When a *brace-or-equal-initializer* is used to initialize a non-static data member, any calls to `current` should correspond to the location of the constructor or aggregate initialization that initializes the member.

[*Note*: When used as a default argument (C++17 §11.3.6), the value of the `source_location` will be the location of the call to `current` at the call site. — *end note*]

[*Example:*

```
struct s {  source_location member = source_location::current();
    int other_member;
    s(source_location loc = source_location::current())
        : member(loc) // values of member will be from call-site
    {}
    s(int blather) : // values of member should be hereabouts
        other_member(blather) {}
    s(double) // values of member should be hereabouts
    {}
};

void f(source_location a = source_location::current()) {
    source_location b = source_location::current(); // values in b represent this line
}

void g() {
    f(); // f's first argument corresponds to this line of code

    source_location c = source_location::current();
    f(c); // f's first argument gets the same values as c, above
}
```

— *end example*]

```
constexpr source_location() noexcept;
```

Effects:

Constructs an object of class `source_location`.

Remarks:

The values are implementation-defined.

11.2.2 `source_location` field access

```
constexpr uint_least32_t line() const noexcept;
```

Returns:

The presumed line number (C++17 §19.8) represented by this object.

```
constexpr uint_least32_t column() const noexcept;
```

Returns:

An implementation-defined value representing some offset from the start of the line represented by this object.

```
constexpr const char* file_name() const noexcept;
```

Returns:

The presumed name of the current source file (C++17 §19.8) represented by this object as an NTBS.

```
constexpr const char* function_name() const noexcept;
```

Returns:

If this object represents a position in the body of a function, returns an implementation-defined NTBS that should correspond to the function name. Otherwise, returns an empty string.

§ 5. Feature macro

Similarly to the original [\[n4417\]](#) wording, we recommend the feature test macro `__cpp_lib_source_location` for this feature.

§ 6. Acknowledgments

The authors would like to thank the people who reviewed early version of this proposal, notably Peter Dimov, Jonathan Wakely, Arthur O'Dwyer and Geoffrey Romer. We would also like to thank Herb Sutter and Niall Douglas for their insightful remarks on `std::error` and errors handling in general.

§ References

§ Informative References

[N4129]

Robert Douglas. [Source-Code Information Capture](https://wg21.link/n4129). 10 October 2014. URL: <https://wg21.link/n4129>

[N4417]

Robert Douglas. [Source-Code Information Capture](https://wg21.link/n4417). 8 April 2015. URL: <https://wg21.link/n4417>

[N4562]

Geoffrey Romer. [Working Draft, C++ Extensions for Library Fundamentals, Version 2](https://wg21.link/n4562). 5 November 2015. URL: <https://wg21.link/n4562>

[N4758]

Thomas Köppe. [Working Draft, C++ Extensions for Library Fundamentals, Version 3](https://wg21.link/n4758). 25 June 2018. URL: <https://wg21.link/n4758>

[P0555R0]

Axel Naumann. [string_view for source_location](https://wg21.link/p0555r0). 20170130. URL: <https://wg21.link/p0555r0>